ISSUE #22 · ARCHITECTURE OVERHAUL · AS BUILT ON wave-6

klint

A small model judge checks claims against evidence: code against rules, and now an AI agent's summaries of its change against the code, from each hunk up to the commit.

Waves 0 to 6 · 17 sub-issues

12 debt items: 9 resolved, 3 reduced

461 tests · 5 import contracts kept

For the director's review of #22 · every claim cites code on wave-6 in the speaker notes

All six objectives are met in code; two leave room by design

#22's objectives in priority order, how each is met, and where.

Objective	Status	How it is met	Where
1. Verify an agent's tiered change summaries, accuracy first	**Met**	One suite and one command check hunk, file, package and commit; accuracy findings come first	`catalog/change_summary/` `cli/change_summary_audit.py`
2. Add suites, units, extractors, rule packs, audit kinds without editing core or app	**Met**	A suite supplies its kinds of audit; a suite written only in tests runs end to end	`core/interfaces/suite.py` `tests/app/test_suite_plugins.py`
3. One use case for the CLI, an API, a web UI, a hook or a GUI	**Met**	AuditPipeline.run_audit serves the three commands and the Stop hook; no API or web UI is built yet	`app/pipeline/audit_pipeline.py`
4. Input and rules from any source	**Ports met**	Source and rule-source ports; adapters today: disk tree, JSON from disk or memory, JSON rule files	`core/interfaces/source.py` `core/interfaces/rule_source.py`
5. Keep the one-way layer chain, checked by import-linter	**Met**	Five contracts, kept at every wave gate	`pyproject.toml`
6. Leave room for finer units and other evidence	**Room left**	Units nest; the renderer picked for each unit class decides what evidence the judge reads	`core/models/unit.py` `core/models/audit_kind.py`

Code: paths under klint/ unless they start with tests/

Five layers; imports flow one way, and a tool checks it

A layer may import only the layers below it. import-linter fails the check on any other import.

cli	Three commands: parse, call the use case, print; exit 0, 1 or 2	`cli/`
app	The use case: one pipeline in five stages; settings; wiring	`app/pipeline/`
catalog	Suites: domain-neutral, code, change summary	`catalog/`
infra	Rule-pack loading; the judge over HTTP or in process	`infra/`
core	Ports, models and the auditor	`core/`

imports

import-linter contracts

Layers: cli > app > catalog > infra > core	KEPT
Suites: app and core import no named suite	KEPT
The domain-neutral suite imports no named suite	KEPT
Named suites do not import one another	KEPT
Language-neutral code does not import one language	KEPT

Code: `pyproject.toml [tool.importlinter]` · `klint/{cli,app,catalog,infra,core}/`

The app's boundaries are ports in core; adapters plug in

Twelve abstract classes in core/interfaces; their adapters live in catalog, infra and cli.

Input

`BaseAuditSourceFactory`

Directory tree from disk; JSON document from disk or memory

`catalog/domain_neutral/sources/`

Extraction

`BaseUnitExtractor[T]`

File contents, project structure, change summaries, whole file

`catalog/*/extraction/`

File traits

`BaseFileTraitsIdentifier`

Language from the extension, tags from the file, klint.json overrides

`catalog/domain_neutral/file_traits_identifiers/`

Rules

`BaseRuleSource`, `BaseRuleLoader`

JSON rule files; rule packs chosen by unit type, language and tags

`infra/rule_loader/`

Evidence

`BaseUnitRenderer`

Seven renderers, one per unit class, within a character budget

`catalog/*/renderers/`

Judge

`BaseJudge`

A System One server over HTTP, or kev in process

`infra/system_one/`

Suites

`BaseSuite`

Domain-neutral, code, change summary

`catalog/`

Progress

`Audit and scan listeners`

The progress bar and skipped-unit lines of the CLI

`cli/`

Code: `klint/core/interfaces/*.py` · `klint/core/models/audit_kind.py (UnitSource)`

Suites are plugins: each one supplies whole kinds of audit

A kind of audit is one AuditKind value. The app looks it up by id and never imports a named suite.

A suite's module

```
multibind(list[BaseSuite], to=
[ChangeSummarySuite])
```

BaseSuite

```
list_audit_kinds()
register_unit_types(registry)
```

AuditKindRegistry

```
find_audit_kind(kind_id); a kind id
offered twice stops the build
```

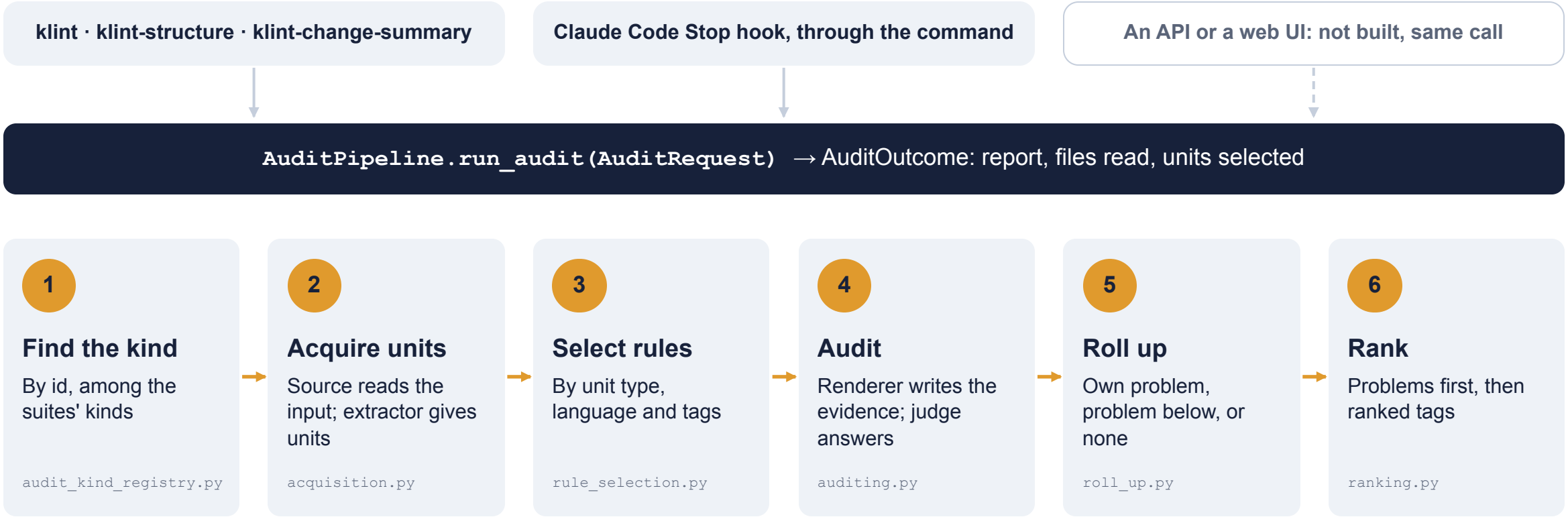
AuditKind field	Code suite: project structure	Change-summary suite
Input	A path on disk	A path on disk, or bytes in memory
Source → extractor	Directory tree → ProjectStructureExtractor	JSON document → ChangeSummaryExtractor
Rules	Built-in packs, chosen by language and tags	summary_checks.json: five checks
Answer scale	Pass · Fail · Irrelevant · Lack of evidence	Accurate · Inaccurate · Cannot tell
Renderers	File metadata, directory, Python package	Hunk, file and package summary
Audit order	All at once	Children first
Ranked tags	None	Accuracy, then completeness

Each row is a field of AuditKind; the last field, the character budget, is 2,000 for both. The domain-neutral suite offers no kind: it registers the file unit the others build on.

Code: klint/core/models/audit_kind.py · klint/core/interfaces/suite.py · klint/app/audit_kind_registry.py

One run, stage by stage, behind one call

Every caller sends an AuditRequest to AuditPipeline.run_audit and gets an AuditOutcome back.



The commands turn the outcome into an exit code: **0** no problem · **1** problems found · **2** the audit could not run, or no unit could be audited

Code: klint/app/pipeline/ (stage files) · klint/app/audit_kind_registry.py · klint/core/auditor.py · klint/cli/exit_code.py

Change summaries are checked level by level, accuracy first

Each hunk's summary is judged against its code; each level above, against the checked summaries below it.

Commit
The root package's summary

↑

Package
Evidence: its files' and packages' checked summaries

↑

File
Evidence: its hunks' checked summaries

↑

Hunk
Evidence: its code, from git diff -U0

Judged deepest first: hunks, files, packages, commit

Five checks

No false statement every level	accuracy
No unsupported reason every level	accuracy
No contradiction with the level below file, package	accuracy
Covers every change every level	completeness
Covers the key changes every level	completeness

Asked as inference: the evidence is the premise, the summary the hypothesis. Accuracy findings are listed first.

Answers

- **Accurate**
- **Inaccurate: a problem**
- **Cannot tell**

Roll-up

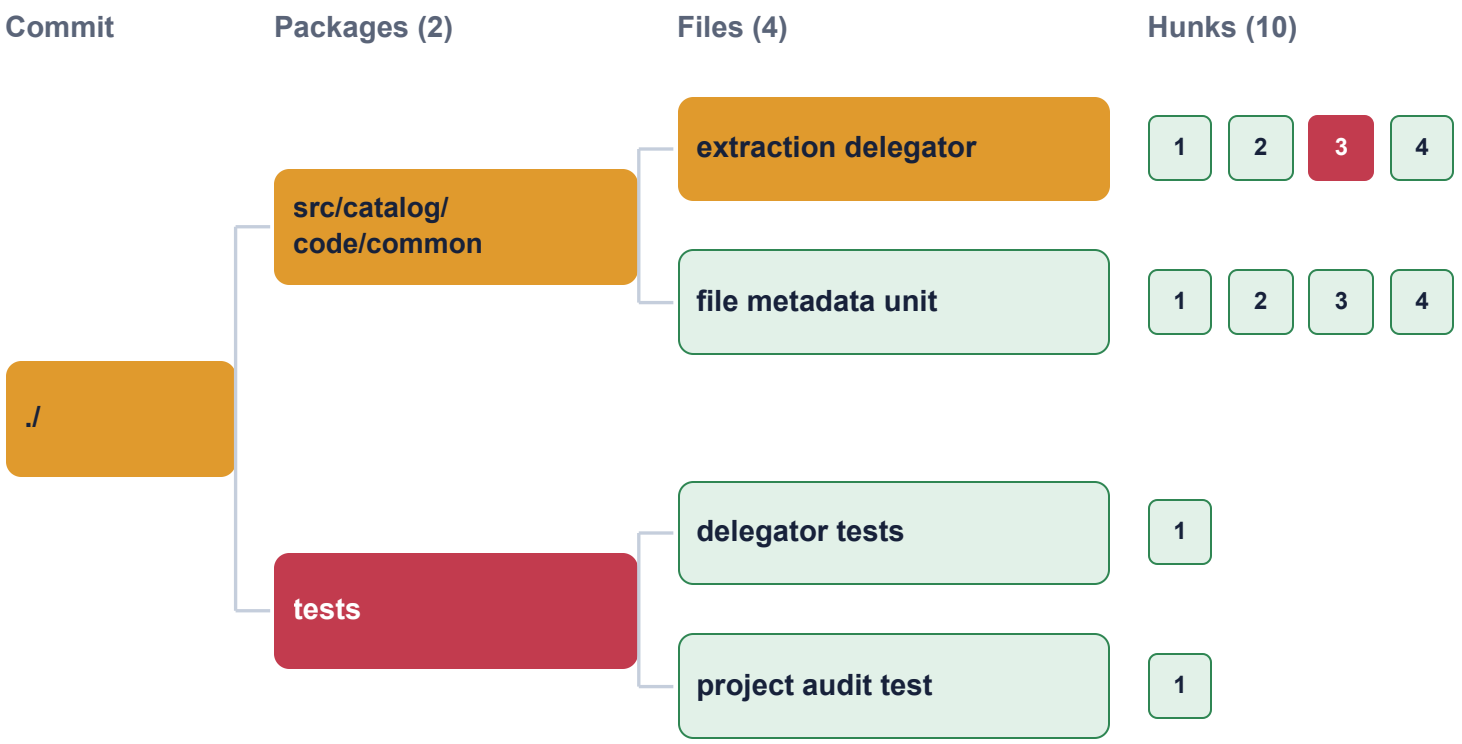
- **OWN PROBLEM**
- **DESCENDANT PROBLEM**
- **NO PROBLEM**

A summary whose own checks pass still shows where a problem sits below it.

`Code: klint/catalog/change_summary/ · klint/resources/catalog/change_summary/summary_checks.json`

Real change: both planted errors caught, no false alarm

Commit d35fa76 of this repository, summarized as an agent would. Live judge: kev-4b.



Planted 1, hunk 3: says the delegator raises ValueError for binary files; the code returns a unit.

Planted 2, package tests: gives a memory reason nothing in the change supports.

2 of 2

planted errors flagged as their own problem

0

other summaries flagged; 9 failing answers, all on the two planted summaries

exit 0 · exit 1

clean document passes; planted document fails, for the Stop hook to block

8%

of answers 'cannot tell' (6 of 75), none on a planted summary

Code: examples/change_summary/ · klint/cli/change_summary_audit.py · live runs in #55, repeated on wave-6

A new suite runs end to end without touching core or app

One test builds a whole suite in test code, adds it next to the built-in ones, and runs it as a command.

1

Write the parts

A unit, an extractor, a renderer, a rule loader and its own answer scale (polite / rude)

```
NoteUnit · NoteExtractor · NoteRenderer · POLITENESS_SCALE
```

2

List one kind of audit

The suite returns one AuditKind from list_audit_kinds()

```
NotesSuite(BaseSuite)
```

3

Add the suite

One binding adds it to the suites the app can draw on

```
multibind(list[BaseSuite], to=[NotesSuite])
```

4

Run it as a command

Exit 1 with a 'Rude' finding on each note, judged on the suite's own scale

```
AuditCommandApp.run([folder, '--json'])
```

0 lines

changed in core or app for a new suite

Wave 5 did it for real: the change-summary suite and its command added 1,181 lines; in core and app only one docstring changed.

Two kinds with different scales run in one process, and a kind id offered twice is refused before any run.

Code: tests/app/test_suite_plugins.py · tests/support/notes_suite.py

Quality is checked by tools at every wave gate

Tests, ruff, ruff format, strict mypy and import-linter pass on wave-6, and a live judge run is recorded per wave.

461

tests pass

Results checked through the commands, the use case and the auditor, with a fake judge

`tests/support/fakes.py`

strict

mypy on klint/ and tests/

No explicit Any allowed anywhere in klint/

`pyproject.toml [tool.mypy]`

291

lines, the largest module

Under the 300-line rule; 146 modules in klint/

`klint/cli/runner.py`

5 of 5

import contracts kept

The layer chain and the suite boundaries

`pyproject.toml`

30 · 18

live failures on the examples

The same before and after the wave 6 renames, field by field

`klint · klint-structure on examples/`

0 · 1 · 2

exit codes

No problem, problems found, could not run: for CI and the Stop hook

`klint/cli/exit_code.py`

Code: pyproject.toml [tool.pytest] [tool.mypy] [tool.importlinter] · tests/ · klint/cli/exit_code.py

The review's debt, re-scored: 269 points down to 32

D1 to D12 from the review of Oct 18, scored again against wave-6. Priority = (impact + risk) × (6 − effort).

ID	Debt in the review	Before	Now	State
D1	Closed audit kinds; app imported a code-suite class	32	**0**	**Resolved**
D2	Judge contract was Mapping[str, Any] both ways	28	**0**	**Resolved**
D4	No end-to-end use case; exit code always 0	28	**0**	**Resolved**
D3	Input was always a path on disk	27	**12**	**Reduced**
D5	Rules chosen by language alone	24	**0**	**Resolved**
D6	One global answer scale and ignore list	24	**0**	**Resolved**
D8	Two extraction families and dispatchers	21	**0**	**Resolved**
D10	Rule loader over the 300-line limit	20	**0**	**Resolved**
D12	No audit order or roll-up	18	**0**	**Resolved**
D7	Prompt text built inside units	16	**12**	**Reduced**
D9	klint.json both config and rules; KEV_ names; mode a string	16	**8**	**Reduced**
D11	Installed package named src	15	**0**	**Resolved**

269 → 32

summed priority · 9 resolved, 3 reduced

New debt found

- Four test files over 300 lines
- Old names to delete next release: KEV_ settings, klint-project, --all, two rule fields
- action.yml inputs still named kev_*
- Setting names are strings, not an enum

Code: see the speaker notes for the file behind each line

What we traded, and what we left for later

Each choice below is in the code today; each deferred item has its port or seam already in place.

Trade-offs we chose

Upper levels read the checked summaries, not the diff

Small context for a 4B judge; a mistake missed below can carry upward

Children first

Each level waits for the one below; units within a level still run together

A small local judge, kev-4b

Cheap and private; 40 to 81% confident on the planted errors; one subtle miss

Suites as plugins with dependency injection

No edit to core or app for a new suite; one registration pattern to learn

A 2,000-character budget per subject

No subject outgrows the judge; a trimmed one says what it left out

Deferred, by design

Git diff, stream, hook and MCP inputs

The source port is ready

An API and a web UI

They call AuditPipeline.run_audit

Finer units and AST extractors for more languages

Units nest; renderers pick the evidence

Kev fine-tuning; the README

Out of scope; the README waits for release

Removing the old names

Next release: delete the aliases and their tests

Code: klint/catalog/change_summary/renderers/ · klint/core/auditor.py · klint/core/interfaces/source.py

RECOMMENDATION

Merge issue-22 into main

Objective 1 works end to end

On a real change, both planted errors are flagged and nothing else is

The design holds its own rules

Five import contracts kept; a new suite needs no edit to core or app

The debt is down to what we chose to leave

9 of 12 items resolved, 3 reduced; 461 tests and strict typing guard it

The issue-22 → main pull request links every sub-issue PR, this deck and the change-summary run.
Deck sources: docs/presentation/source/ · speaker notes: docs/presentation/speaker_notes.md